

有限電池 × 變動服務窗口

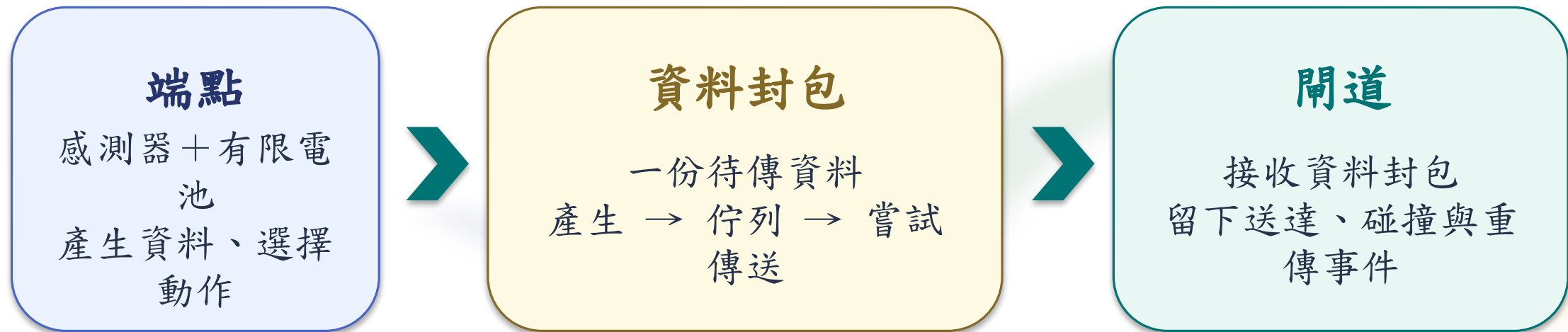


先看窗口，再看動作；最後讀服務與端點能量

scenario (情境定義) | 來源: course package | 作用: 固定端點、窗口與資料規則
單位: identifier | 判讀: 相同輸入可重跑

LoRaEnergySim 是什麼

把端點的一次選擇，放進可重複的封包與能量模擬



它回答：同一份工作，在等待、休眠或傳送的不同決策下，服務與端點能量（J）如何一起變？

simulator（模擬器） | 來源：course package 參照 LoRaEnergySim | 作用：在固定時鐘重播端點、封包與能量假設
單位：一次固定執行 | 判讀：產生可比較證據，不是實測

原始上游套件已做什麼

上游研究模型提供「端點狀態 → 封包事件 → 能量」的可重用能力

上游已有的研究能力

休眠／處理／發射／接收狀態

封包、碰撞、重傳、端點能量

上游沒有替本課固定的部分

固定情境、LEO 服務窗口、案例順序

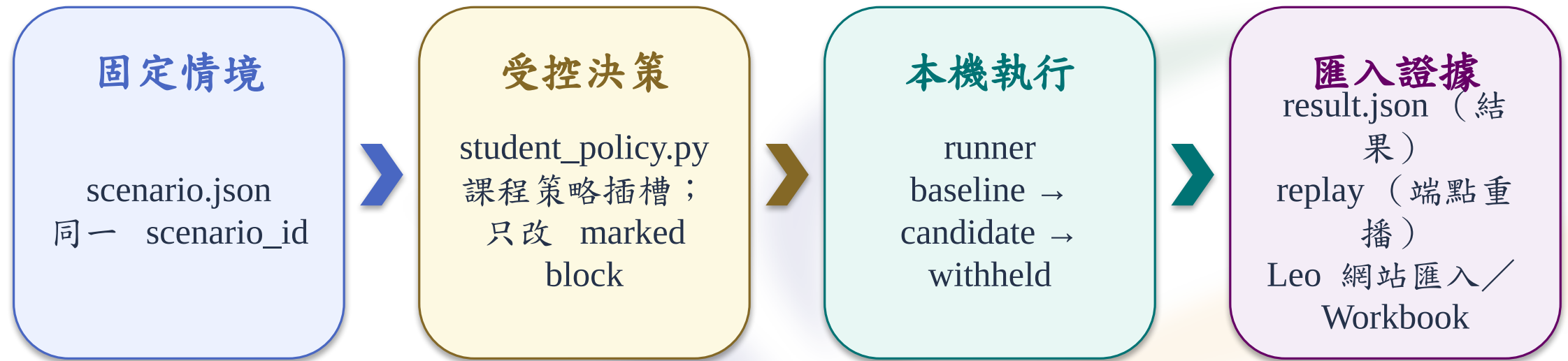
受控策略、結構化匯入、Leo 重播

因此：上游能力是參照；課程流程、案例與證據邊界由本課封裝。

`upstream_model`（上游模型能力） | 來源：GillesC/LoRaEnergySim@f854462c | 作用：提供端點狀態、封包與能量概念
單位：能力集合 | 判讀：僅是上游能力，不是本課 runner 執行

課程封裝補了什麼

把研究模型包成一條可教、可驗、可匯入的固定路徑



student_policy.py 是課程策略插槽； runner 不直接執行上游 LoRaEnergySim，上游僅能力參照。

runner (課程執行器) | 來源：course package | 作用：讀固定 scenario、載入 bounded policy、輸出 result / replay
單位：run artifact | 判讀：把一次修改連到可重播證據

為什麼競賽要用它

先得到可重複的因果證據，再回到 LEO 網站解讀

可重複

同一情境、相同策略檔
與固定種子
重跑同一條事件路徑

可檢驗

改一個受控區塊
檢查封包／服務／狀態
／J 是否改變

可轉移

智慧農業、暖通空調、
物流
重訂窗口、期限與能量
邊界

本套件不是即時資料、典範系統指標或整體 LEO 能量；它先建立可反駁的端點因果證據。

evidence（證據） | 來源：result.json + endpoint replay | 作用：保存 policy → event → service / J 的路徑
單位：record | 判讀：支持 bounded comparison，不擴大 claim

只有 run case 才產生結果檔

setup / verify 只檢查環境；run case 才會寫入一次執行的產物



只有 run case 會產生 artifacts/<run_id>/result.json 與 endpoint-replay.json；setup / verify 不算。

run artifact (執行產物) | 來源：本機 runner | 作用：保存一次執行的 result 與 endpoint replay
單位：同一 run 目錄 | 判讀：只有 run 產生；setup / verify 不算

課程頁 /course 只匯入結果檔

本機 runner 做模擬；網站做驗證、具象化、重播與工作簿

本機 runner（模擬）

Linux / macOS 終端機
WSL 也使用 Linux 指令
讀 student_policy.py
（課程策略插槽）



/course（課程頁）只選 result.json

驗證

具象化

重播／工作簿

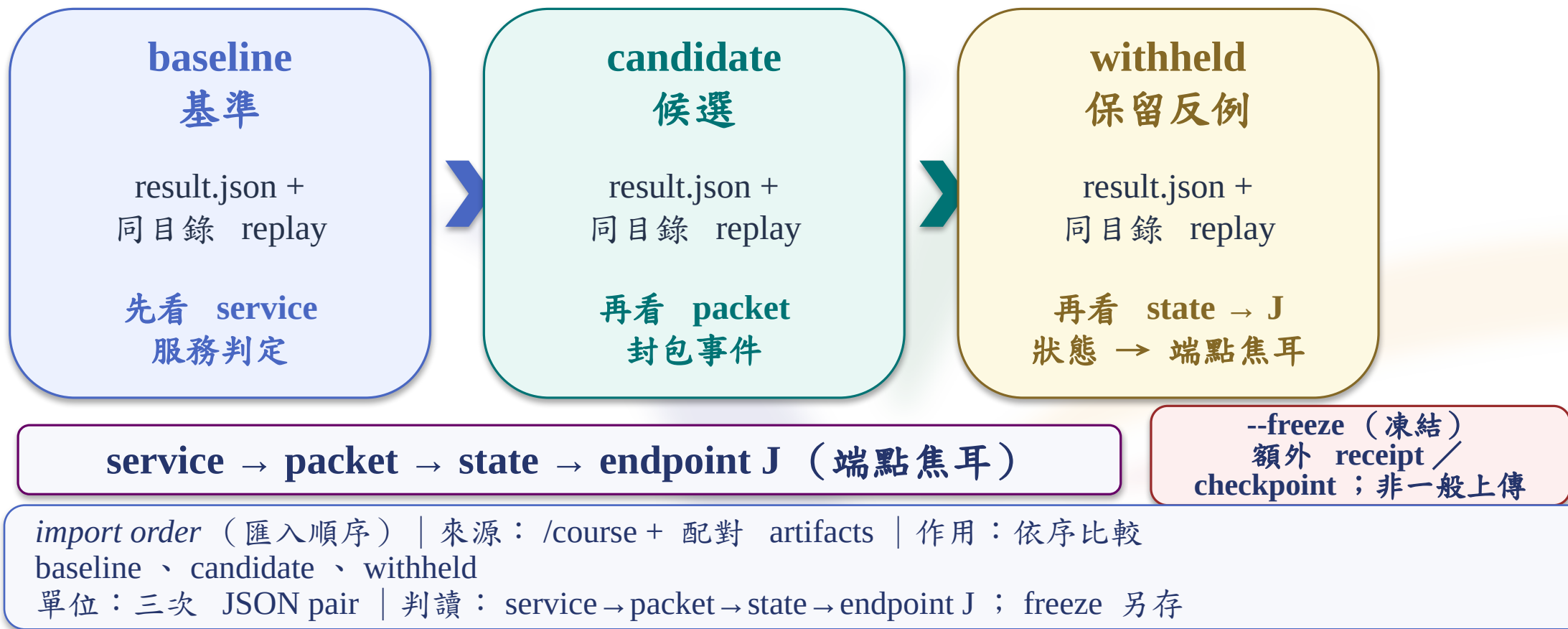
endpoint-replay.json 留在同一目錄配對
不選、不上傳 student_policy.py

policy 原始檔只在本機 runner 讀取；/course 接收 result.json，不接收 Python。

course import（課程匯入） | 來源：/course 控制項 | 作用：驗證 result.json，建立重播與工作簿
單位：JSON pair | 判讀：選結果檔；replay 留同目錄

三次匯入，沿同一解讀順序

baseline → candidate → withheld：每次匯入一對檔案，再按同一順序解讀



一次動作，同時改變服務與能量

同一個 *action* 會沿不同路徑留下可觀察後果

SLEEP | 低功耗休息

休息功率降低
喚醒延遲與能量增加

WAIT | 清醒等待

反應能力保留
清醒閒置能量累積

SEND | 送出封包

服務機會增加
處理與無線事件增加

服務結果

端點能量 (J)

action (策略動作) | 來源: policy API | 作用: 每次決策的唯一輸出
單位: enum | 判讀: 觸發狀態與封包轉移

先讀中間事件，再讀最終結果

能量或服務差異，必須由中間事件支持



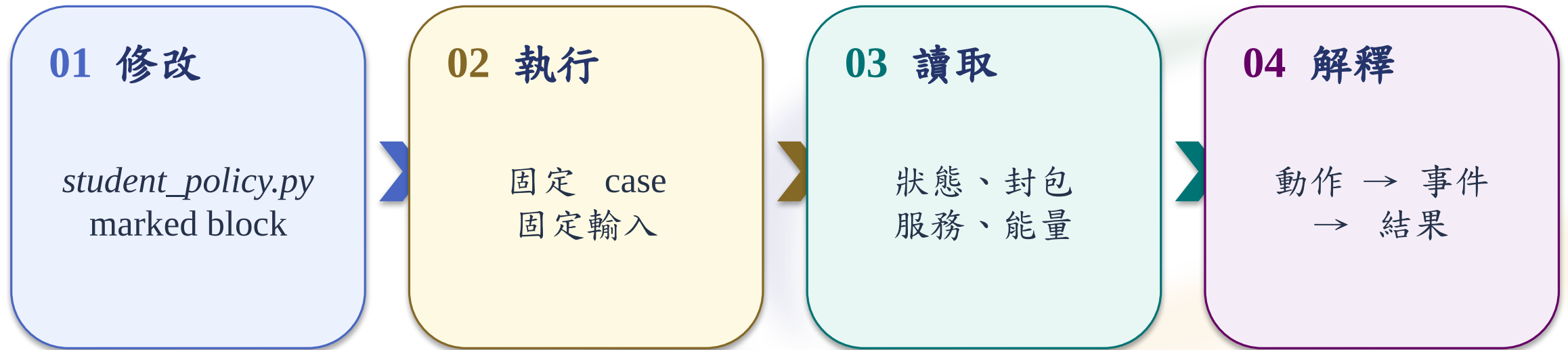
窗口關閉 → 傳輸類動作不執行 → 安全分支為 *SLEEP*

若中間事件沒有差異，最後的數值變化不得歸因於本次修改。

radio_state（無線狀態） | 來源：runner event ledger | 作用：表示各狀態區間
單位：state enum + duration | 判讀：提供端點能量的時間分段

固定情境中的四步操作閉環

只改一個受控常數，再沿同一條證據路徑比較

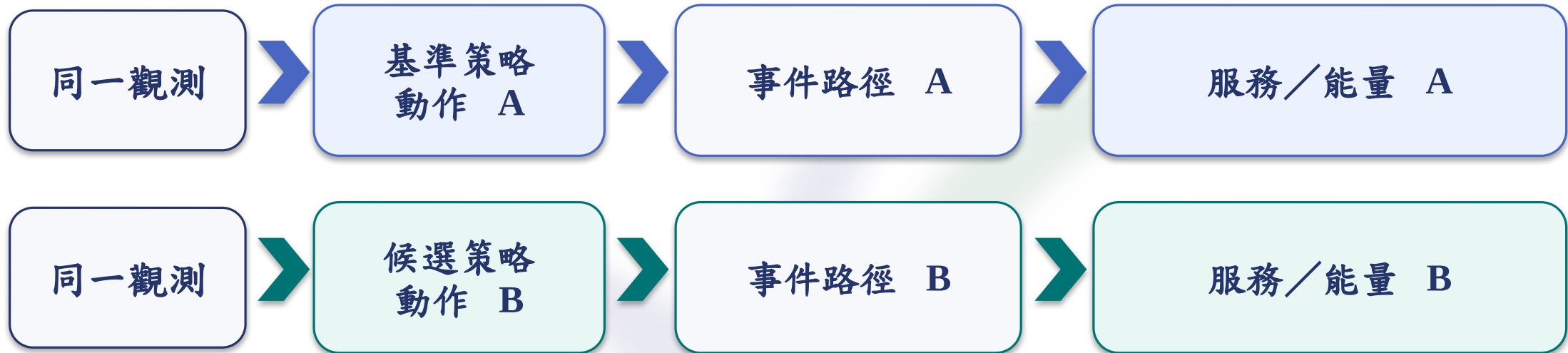


預測與結果同時保存；反例保留為可檢驗證據。

`result.json`（結果資料檔） | 來源：package runner | 作用：保存 run identity、狀態、封包、服務與能量
單位：JSON artifact | 判讀：作為 replay 與 import 的輸入

中間事件是因果歸因的必要橋樑

同一觀測下，比較基準與候選策略的事件路徑



先確認事件路徑不同，才比較服務與能量。

`packet_progress`（封包進度） | 來源：result / replay artifact | 作用：記錄產生、佇列、嘗試、重試、送達與逾期
單位：event count + time | 判讀：連接策略動作與服務結果

Runner 證據有明確邊界

可重跑，不等於可以外推到所有能量層級

本套件可支持

固定 scenario / seed
端點狀態與封包事件
服務判定與端點 J

本套件不支持

LEO / system energy
canonical efficiency
實測或即時 KPI

endpoint_energy_j (端點能量) | 來源: endpoint radio / processing model | 作用: 累積狀態能量
單位: J | 判讀: 只屬於 endpoint evidence layer

LEO 是變動服務窗口的工作範例

LEO 改變可服務機會；主要控制仍在端點策略



相同機制可以轉移；窗口、期限與能量邊界必須重新定義。



`service_window`（服務窗口） | 來源：fixed scenario trace | 作用：定義合法傳輸區間
單位：trace time | 判讀：決定動作是否具有服務機會

從預測到結果，保留可反駁路徑

先寫下方向，再執行；結果不符時保留反例



保留條件案例維持 frozen policy，用來檢查機制是否可轉移。

prediction（預測） | 來源：workbook record | 作用：指定可反駁的變化方向
單位：statement | 判讀：作為執行前的比較基準

三個實驗到底改什麼

三次都只改 `student_policy.py` 的一個 `marked block` ；
`runner` 、 `scenario` 、 `schema` 不動

Lab A | 休息方式

SLEEP → WAIT
PACE_GAP_STEPS = 2 不變

觀察：SLEEP /
AWAKE_IDLE / WAKE 的時間與 J；封包服務

Lab B | 穩定門檻

STABLE_STEPS : 2 → 1
ENTER_QUALITY=2 /
EXIT_QUALITY=1 不變

觀察：進入時機、attempt /
retry / delivery、service、J

Lab C | 急件提前量

URGENT_MARGIN_S :
20 → 5 → 30
BATCH_SIZE = 3 不變

觀察：urgent
timing、deadline / expired、
service、J

同一檔案 / 同一 `scenario` / 固定 `runner` → `prediction` → `baseline` → `candidate` → `withheld`

marked block（受控區塊） | 來源：`student_policy.py` | 作用：限制可修改的最小決策面
單位：one block | 判讀：其他行與檔案不動，才能歸因

LoRaEnergySim 提供的三項價值

它提供成熟的 LoRa 端點問題框架，讓狀態、封包與能量能沿同一條路徑解讀

端點狀態與能量

把休眠、處理、TX、RX
連到端點能量邊界

封包事件與可靠度

用封包、碰撞、重試、送
達
解釋策略後果

可重複研究架構

固定模型來源與版本
讓案例與結果可以比較

LoRaEnergySim 讓策略動作可以連到狀態、封包、服務與端點能量證據

model_reference（模型參考） | 來源：固定的 LoRaEnergySim repository 與版本 | 作用：提供端點狀態、封包事件與能量欄位的可追溯來源
單位：來源識別 | 判讀：以相同模型語言比較不同策略的事件與結果

把下載資料搬進 Linux 課程資料夾

這一步只做路徑準備：把 release 帶進 Linux，再確認程式根目錄

① 這個工作區

WSL = Windows 內的 Linux
讀
`/mnt/c/.../Downloads`

② 程式不改

`student_policy.py` 不動
只複製 release 副本

③ 進入根目錄

下一步集中到
Linux
`lora-energy-lab/`

④ 核對結果

執行 `pwd` 只讀位置
路徑不對就先停止

先把下載副本放進 Linux，再確認根目錄；路徑錯會讓後續設定讀錯檔

`pwd`（目前目錄命令） | 來源：Linux / macOS 終端機（WSL 也使用 Linux 指令） | 作用：確認目前位置
單位：路徑文字 | 判讀：輸出應位於 Linux home 的課程資料夾

建立環境並確認 READY

這一步建立可重現環境；沒有程式策略改動，先拿到 READY

① Linux / macOS 終端機

WSL 也使用 Linux 指令

② 建立環境

執行 `setup.sh`
不改 `policy source`

③ 下一步

執行 `course.sh verify`
預期看到 `READY`

④ Windows shells

PowerShell /
Command Prompt
只讀 `READY` 結果

指定版本 → 建立隔離環境 → `READY`；此時尚無案例結果

`PYTHON_BIN`（Python 執行器路徑變數） | 來源：命令工具設定 | 作用：指定要使用的版本
單位：路徑文字 | 判讀：供建立環境的命令讀取

READY 只代表環境可用

這一步只讀環境收據；它不是案例結果，也不會改程式

① Linux / macOS 終端機

WSL 也使用 Linux 指令
只讀收據

② Windows PowerShell

只讀同一份收據
不修改檔案

③ Windows Command Prompt

只讀同一份收據
不修改檔案

④ 下一步 / 看什麼

核對狀態、版本、身份
一致才進案例

不同工具只是在讀同一張收據；檔案內容與來源不變

`verify-receipt.json`（驗證收據檔） | 來源：環境檢查命令 | 作用：記錄環境與契約關卡
單位：JSON 檔案 | 判讀：只解讀環境證據，不代替案例結果

確認環境與案例身份

這一步確認能不能進案例，以及案例是否綁定正確情境

① 狀態

status 必須 **READY**
否則不能 **run**

② 版本

Python 必須 **3.11.x**
不改 **policy**

③ 情境

固定案例身份
scenario_id

④ 下一步／證據

一致才執行 **run**
結果另看 *result.json*

READY 只代表環境可用；服務與能量要等案例結果

status（狀態） | 來源：驗證收據 | 作用：表示關卡是否可進入案例
單位：狀態值 | 判讀：通過只表示環境與契約一致

說清資料來源與主張邊界

這一步標明資料來源；主張邊界比漂亮數字更重要

① 資料模式

課程使用模擬資料
engine_mode

② 是否實跑

沒有執行上游程式
false

③ 不可超出

非即時、非實測
不叫 *canonical*

④ 下一步／證據

run 另產生 *identity*
讀 *artifact_source*

先標資料來源，再決定能說到哪裡；不能把模擬說成實測

engine_mode（執行模式） | 來源：執行器紀錄 | 作用：分類資料產生方式
單位：模式值 | 判讀：模擬資料不延伸成上游程式執行

卡住時修復或保留來源

這一步處理卡關：修得好就重跑，修不好也不冒充本機執行

① 正常

READY 後執行案例
產生新結果

② 修復什麼

只修
Python、root、lock
不改 policy block

③ 回復什麼

同情境結果成對保存
result.json / *endpoint-replay.json*

④ 看什麼

讀 *artifact_source*
判斷本機或回復

修復不改策略；回復不冒充本機執行

artifact_source（資料來源分類） | 來源：結果紀錄 | 作用：標示本機執行或回復資料
單位：來源值 | 判讀：來源模式保持原值，不改寫成本機執行

只可修改三個標記區塊

這一步畫出可編輯邊界：三個實驗區可改，其他程式只讀

① 唯一檔案

只在這裡做實驗
`student_policy.py`

② 可改區段

A / B / C 三個標
記
marked block

③ 保持不變

中文註解、函式、情
境
runner、schemas 不
動

④ 下一步／證據

只開目前 lab
結果仍看 result /
replay

只開目前實驗區；其他檔案與輸出契約保持不變

marked block（標記區段） | 來源：檔案標記 | 作用：界定一個實驗的可編輯範圍
單位：程式碼區段 | 判讀：其他套件邊界維持唯讀

程式每次判斷會看什麼

這一步先說清楚輸入：每個 step 只看目前／過去資料

① 讀目前資料

窗口、急件、間隔、
品質
都是當下 snapshot

② 不讀未來

不看結果摘要
不回看 energy

③ 交給函式

把目前資料交給
`choose_action()`
再回一個合法動作

④ 下一步／證據

回一個合法動作
runner 再記 event /
result

當下觀察 → 合法動作；這一頁沒有程式值變更

`choose_action`（策略函式） | 來源：student_policy.py | 作用：依當下觀察回傳一個合法動作
單位：函式契約 | 判讀：只讀目前／過去欄位，不讀未來結果

判斷順序如何產生動作

這一步把合法動作順序翻成白話；鏡像可讀，不能貼回程式

① 窗口關閉

不能送出，先休息
SLEEP

② 期限逼近

急件先送出
SEND_URGENT

③ 節奏／品質

間隔未到休息；品質未
穩等待
REST_DURING_GAP /
WAIT

④ 佇列／保底

夠量批次，否則單筆；
無事等待
FLUSH_BATCH /
SEND_ONE / WAIT

先看窗口，再看期限、間隔、品質與佇列；順序會改變動作

action（合法動作） | 來源：policy API | 作用：把條件轉成 runner 可執行的動作
單位：enum | 判讀：下一步從 state / packet / service / endpoint J 讀後果

縮排與回傳決定程式走哪條路

這一步只讀一小段程式：看縮排如何選分支，再把動作交回執行器

① 先看意思

窗口關閉就不送
`if ...: return SLEEP`

② 縮排做什麼

把條件放進正確分支
只修 active block

③ 先檢查

執行 `py_compile`
只檢查語法、不改檔

④ 下一步／看什麼

通過後跑 baseline
讀 result / replay
pair

縮排 → 分支 → 回傳 → runner 事件；語法檢查不改內容

`return`（回傳） | 來源：policy function | 作用：把一個合法動作交給執行器
單位：動作值 | 判讀：語法通過後再看 result / replay 的實際事件

先建立基準結果再比較

這一步先保存沒有改動的基準，下一步才比較候選值

① 固定輸入

packaged policy + 固定情境
不讀未來結果

② 執行命令

只跑 baseline
`bash course.sh run --lab A --case baseline`

③ 產生結果

新增 result / replay pair
原始程式不回寫

④ 下一步／看什麼

確認 status: OK、路徑先 service，再成本

先保存基準，再凍結規則，最後比較候選；結果不能回頭改 block

`result_path`（結果檔路徑） | 來源：命令輸出 | 作用：定位 `result.json`
單位：路徑文字 | 判讀：網站只匯入 `result.json`；`endpoint replay` 只做配對，`runner` 不是網站

先找到唯一要改的地方

Lab A 比較空檔休眠與清醒等待，觀察服務與端點能量的取捨

檔案

student_policy.py
只修改這一份策略檔

標記區塊

lab-a-pace-rest
只開目前 Lab

唯一變數

REST_DURING_GAP
其他值與檔案不動

保持不變：PACE_GAP_STEPS = 2、runner、scenario、schemas 與其他 Lab 區塊

REST_DURING_GAP（空檔動作） | 來源：student_policy.py marked block | 作用：控制本 Lab 的單一策略差異
單位：設定值 | 判讀：只有此值可改，才能把結果差異歸因於本次修改

只改這一行：SLEEP → WAIT

兩邊程式結構完全相同；只比較反白的一個值

修改前 | Before

REST_DURING_GAP = SLEEP

空檔進入低功耗休息

修改後 | After

REST_DURING_GAP = WAIT

空檔保持清醒等待

唯一修改：SLEEP → WAIT；不是修改 PACE_GAP_STEPS

REST_DURING_GAP（本次修改值） | 來源：目前 Lab 的 marked block | 作用：改變策略判斷
單位：依變數定義 | 判讀：修改後先做語法檢查，再執行指定 case

為什麼 SLEEP → WAIT，接著怎麼執行

WAIT 可能減少再次喚醒，卻會累積清醒閒置能量；必須同時看服務與焦耳

SLEEP

空檔進入低功耗
之後傳送要 WAKE

WAIT

空檔保持清醒
awake-idle 持續累積

封包後果

動作時機改變
attempt / retry /
delivery

最後判斷

先 service
再比較 endpoint J

`bash course.sh run --lab A --case baseline` → `bash course.sh run --lab A --case candidate --freeze` → `bash course.sh run --lab A --case hidden`

REST_DURING_GAP（空檔動作） | 來源：Lab A marked block | 作用：決定送出間隔中的休息方式
單位：action | 判讀：Windows 將 `bash course.sh` 改為 `course.cmd`，其餘參數不變

上傳後怎麼解讀

每次只上傳該 run 的 result.json；先確認服務，再解釋原因與能量

網站操作

- ① 依 stdout 的 result_path 找檔案
- ② /course 選擇 result.json
- ③ 保留同目錄 endpoint-replay.json
- ④ 核對 scenario ID 與 run ID

如何回答結果

預期先在 sleep / awake-idle / wake 時間看到差異，再檢查封包服務與端點焦耳。

若沒有看到預期事件差異，就不能把最終數值歸因於這次修改。

① 服務
是否完成任務



② 封包
送達 / 重試 / 逾期



③ 狀態
休眠 / 清醒 / 傳送



④ 端點能量
最後才讀焦耳

radio_state（無線狀態） | 來源：result / replay artifact | 作用：支持本 Lab 的機制判讀
單位：依欄位定義 | 判讀：WAIT 是否真的改變狀態分布，必須由 replay 支持

先找到唯一要改的地方

Lab B 比較品質需要連續穩定幾步，觀察進入可送模式的時機

檔案

student_policy.py
只修改這一份策略檔

標記區塊

lab-b-enter-exit-hold
只開目前 Lab

唯一變數

STABLE_STEPS
其他值與檔案不動

保持不變：ENTER_QUALITY = 2、EXIT_QUALITY = 1、runner、scenario、schemas 與其他 Lab 區塊

STABLE_STEPS（穩定步數） | 來源：student_policy.py marked block | 作用：控制本 Lab 的單一策略差異
單位：設定值 | 判讀：只有此值可改，才能把結果差異歸因於本次修改

只改這一行：2 → 1

兩邊程式結構完全相同；只比較反白的一個值

修改前 | Before

`STABLE_STEPS = 2`

需要連續兩個穩定觀察

修改後 | After

`STABLE_STEPS = 1`

一個穩定觀察就成立

唯一修改：2 → 1；進入與退出品質門檻都不變

`STABLE_STEPS`（本次修改值） | 來源：目前 Lab 的 marked block | 作用：改變策略判斷
單位：依變數定義 | 判讀：修改後先做語法檢查，再執行指定 case

為什麼 2 → 1，接著怎麼執行

需要的穩定觀察變少，可能更早進入可送模式，也可能對短暫品質變化過度反應

原本需要兩次穩定觀察

第 1 步看到品質達標：先等待
第 2 步仍達標：才讓 `quality_ready` 成立

改成一次穩定觀察

第 1 步看到品質達標：立刻讓
`quality_ready` 成立
`MODE_CHANGE` 可能提早出現

```
bash course.sh run --lab B --case trace-a-baseline
```

```
bash course.sh run --lab B --case trace-a-candidate --freeze → bash course.sh run --lab B --case trace-b
```

`STABLE_STEPS`（穩定步數） | 來源：Lab B marked block | 作用：決定品質達標要連續維持幾個 step

單位：steps | 判讀：Windows 將 `bash course.sh` 改為 `course.cmd`；Trace B 保持 frozen policy

上傳後怎麼解讀

每次只上傳該 run 的 result.json；先確認服務，再解釋原因與能量

網站操作

- ① 依 stdout 的 result_path 找檔案
- ② /course 選擇 result.json
- ③ 保留同目錄 endpoint-replay.json
- ④ 核對 scenario ID 與 run ID

如何回答結果

預期 quality_ready 或 MODE_CHANGE 可能提早；再看 attempt、retry、delivery、service 與端點焦耳。

若沒有看到預期事件差異，就不能把最終數值歸因於這台機器

① 服務
是否完成任務



② 封包
送達／重試／逾期



③ 狀態
休眠／清醒／傳送



④ 端點能量
最後才讀焦耳

MODE_CHANGE（模式變更事件） | 來源：result / replay artifact | 作用：支持本 Lab 的機制判讀
單位：依欄位定義 | 判讀：Trace A 的改善必須再由 frozen policy 的 Trace B 檢查

先找到唯一要改的地方

Lab C 比較急件要提前多久送出，觀察期限、服務與能量代價

檔案

student_policy.py
只修改這一份策略檔

標記區塊

lab-c-batch-urgent
只開目前 Lab

唯一變數

URGENT_MARGIN_S
其他值與檔案不動

保持不變： `BATCH_SIZE = 3` 、 `runner` 、 `scenario` 、 `schemas` 與其他 Lab 區塊

URGENT_MARGIN_S（急件提前量） | 來源： *student_policy.py* marked block | 作用：控制本 Lab 的單一策略差異
單位：設定值 | 判讀：只有此值可改，才能把結果差異歸因於本次修改

基準值 | Baseline : URGENT_MARGIN_S = 20

這一頁只讀一個值； BATCH_SIZE = 3 與其他程式全部不變

基準值 | Baseline

URGENT_MARGIN_S = 20

期限剩 20 秒時進入急件分支

只改急件提前量；先預測 SEND_URGENT 時機，再執行對應 case

URGENT_MARGIN_S（急件提前量） | 來源：Lab C marked block | 作用：決定剩餘期限多早進入急件分支

單位：秒 | 判讀：數值越大代表越早嘗試急件傳送；是否有利仍由服務與事件證據判斷

候選值 | Candidate : URGENT_MARGIN_S = 5

這一頁只讀一個值； BATCH_SIZE = 3 與其他程式全部不變

候選值 | Candidate

URGENT_MARGIN_S = 5

等到只剩 5 秒才急送，可能太晚

只改急件提前量；先預測 SEND_URGENT 時機，再執行對應 case

URGENT_MARGIN_S（急件提前量） | 來源：Lab C marked block | 作用：決定剩餘期限多早進入急件分支

單位：秒 | 判讀：數值越大代表越早嘗試急件傳送；是否有利仍由服務與事件證據判斷

修訂值 | Revision : URGENT_MARGIN_S = 30

這一頁只讀一個值； BATCH_SIZE = 3 與其他程式全部不變

修訂值 | Revision

URGENT_MARGIN_S = 30

期限剩 30 秒就急送，較早但可能增加成本

只改急件提前量；先預測 SEND_URGENT 時機，再執行對應 case

URGENT_MARGIN_S（急件提前量） | 來源：Lab C marked block | 作用：決定剩餘期限多早進入急件分支

單位：秒 | 判讀：數值越大代表越早嘗試急件傳送；是否有利仍由服務與事件證據判斷

為什麼 5 秒可能太晚，30 秒可能更早

急件門檻控制的是『何時放棄等待批次、改成優先送出』

20 秒 | 基準

在期限剩 20 秒時
允許 SEND_URGENT

5 秒 | 候選

等待更久才急送
可能錯過窗口或期限

30 秒 | 修訂

更早進入急件分支
可能恢復服務，也可能增加能量

門檻 → SEND_URGENT 時機 → wake / TX / delivery / deadline → service → endpoint J

deadline_pass（期限是否通過） | 來源：result service summary | 作用：確認必要封包是否在期限內完成
單位：布林值 | 判讀：先確認期限與服務，再比較端點焦耳；較低焦耳但失敗不是改善

四次執行的固定順序

每次先設定正確值，再執行對應 case ; revision 才使用 freeze

① baseline

值為 20
`--lab C --case baseline`

② candidate

改成 5
`--lab C --case candidate`

③ revision

改成 30 並凍結
`--lab C --case revision`
`--freeze`

④ surprise

保持 frozen 30
`--lab C --case surprise`

Linux / macOS / WSL : `bash course.sh run ...` Windows : `course.cmd run ...` ; 每次保存 stdout 的 `result_path`

`freeze` (策略凍結) | 來源: revision 執行選項 | 作用: 保存要帶入 surprise 的策略身分
單位: 狀態 | 判讀: `freeze` 缺失或身分不一致時停止 surprise, 不重新修改策略

上傳後怎麼解讀

每次只上傳該 run 的 result.json；先確認服務，再解釋原因與能量

網站操作

- ① 依 stdout 的 result_path 找檔案
- ② /course 選擇 result.json
- ③ 保留同目錄 endpoint-replay.json
- ④ 核對 scenario ID 與 run ID

如何回答結果

先確認 deadline 與 service；若 20 與 30 結果相同，保留『此案例無差異』，不要硬造改善。

若沒有看到預期事件差異，就不能把最終數值歸因於這次修改。

① 服務
是否完成任務



② 封包
送達／重試／逾期



③ 狀態
休眠／清醒／傳送



④ 端點能量
最後才讀焦耳

deadline_pass（期限是否通過） | 來源：result / replay artifact | 作用：支持本 Lab 的機制判讀
單位：依欄位定義 | 判讀：5 秒失敗或 30 秒恢復都要由本次 result / replay 證據支持